# KAREL THE ROBOT

First, let's talk about the **actions** Karel can take, i.e., what he can do. They are listed below:

| | |
|---|---|
| `move()` | Karel moves one tile forward in the direction he is facing. If he is in front of a wall, he crashes into it and this will end the program. |
| `turn_left()` | Karel turns in-place counter-clockwise by 90 degrees. He remains on the same tile. |
| `turn_right()` | Similar to turn_left, but now clockwise by 90 degrees. |
| `take_item()` | Karel picks up one item from the tile he is standing on. If there is no item there, it will end the program. |
| `put_item()` | Karel takes one item from his bag and places it on the tile he is standing on. If his bag is empty, it will end the program. |
| `turn_off()` | Karel turns himself off and checks if he accomplished his end-goal. |

Next, let's look at specific **queries** Karel can execute to learn about what is going on in his surroundings. You can think of each of these resulting in a "yes" or "no" answer.

| | |
|---|---|
| `wall_in_front()` | Is there a wall between Karel and the next tile (in the direction Karel is facing)? |
| `wall_to_left()` | Similar to wall_in_front(), but now looking to the left (so 90 degrees counterclockwise from direction Karel is facing). |
| `wall_to_right()` | Similar to wall_to_left(), but now looking to the right. |
| `item_present()` | Are there one or more items on the tile Karel is standing on? |
| `bag_empty()` | Is Karel's bag empty (i.e., are there no items in his bag)? |
| `facing_north()` | Is Karel facing north? |
| `facing_east()` | Is Karel facing east? |
| `facing_south()` | Is Karel facing south? |
| `facing_west()` | Is Karel facing west? |

# VARIABLES

```
int main() {

    int moves;                  // Declaration
    int num_items;              // Declaration

    moves = 5;                  // Assignment
    num_items = 0;              // Assignment
}
```

## 3.1.3 Naming Conventions

When choosing the names of your variables in C, there are a few rules that must be followed, or you will get an error when you try to compile your code:

1. Variable names must start with either a letter or an underscore ( _ ).
2. Variable names can only contain letters, underscores, and numbers.
3. Variable names cannot contain spaces; instead use an underscore to separate words.
4. Only the first 31 or 63 characters of a variable name count[1].
5. Variable names cannot be one of the 32 reserved keywords in C (i.e., words that have a special meaning in the language). These are listed in the table below. At this point, don't worry about what all of these are; just understand that your variable names cannot be one of these keywords.

| | | | |
|---|---|---|---|
| auto | double | int | struct |
| break | else | long | switch |
| case | enum | register | typedef |
| char | extern | return | union |
| const | float | short | unsigned |
| continue | for | signed | void |
| default | goto | sizeof | volatile |
| do | if | static | while |

```
int main() {
    int x = 4, y = -4, z;

    if ( x )        // x evaluates to 4, therefore expr is true
        z = 1;      // The if-block is executed

    if ( 1 )        // expr is true
        z = 2;      // The if-block is executed

    if ( x + y )    // x+y evaluates to 0, expr is false
        z = 3;      // The if-block is not executed
}
```

This true and false are **logical** values. An expression doesn't actually return the words true or false. Rather, an expression results in a numerical value, as we saw before. However, in constructs where logical values are needed (like if-statements, while-loops, etc.), C interprets numerical values as logical values in the following way[3]:

**Zero** is interpreted as **false**

**Anything else** is interpreted as **true**

| Operator | |
|---|---|
| Logical NOT | `!` |
| Logical OR | `\|\|` |
| Logical AND | `&&` |

| Truth table | | | |
|---|---|---|---|
| *!T = F* | *!F = T* | | |
| *F\|\|F = F* | *F\|\|T = T* | *T\|\|F = T* | *T\|\|T = T* |
| *F&&F = F* | *F&&T = F* | *T&&F = F* | *T&&T = T* |

```
int main() {

    int x = 2, y;

    if ( x == 4 )           // x != 4, expr. is false
        y = 1;

    if ( x = 4 )            // x is assigned the value 4
        y = 1;              // This evaluates as true !!!
}
```

```
int main() {
   int i, x = 1;

   for (i = 0; i < 5; i++) {
      x = x * 2;
   }

}
```

```
int main() {
   int i, x = 1;

   i = 0;
   while (i < 5) {
      x = x * 2;
      i++;
   }
}
```

```
#include <karel.h>

// Function definition
// The function returns the number of items collected
int collect_items() {
    int x = 0;
    while (item_present()) {
        take_item();
        x++;
    }
    return x;                       // Returns the value of x
}

int main() {
    karel_setup("settings/settings00.json");

    int num_items;
    num_items = collect_items();  // Function call
}
```

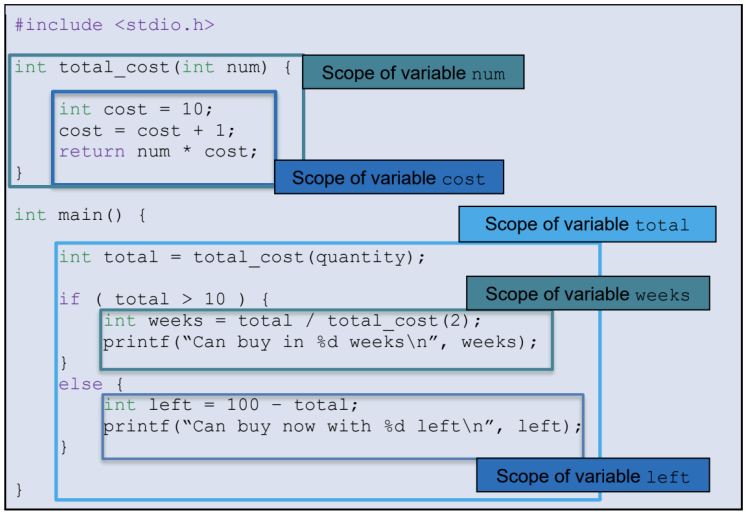
```
#include <stdio.h>

int main() {

    int x = 3;
    int y = 4;
    printf("%d + %d = %d\n", x, y, x+y);
}
```

```
3 + 4 = 7
[User]:~:$
```

The **`#define`** keyword is used to declare a constant in your program and is placed at the top of the program outside of any functions. It is now easier to understand the code. It is also easier to change the value everywhere in our program, since we now only would need to modify the definition of the constant. The syntax to define a constant is shown below. Note that there is no "**=**" as in variable assignment.

```
#define CONSTANTNAME value
```

As a result, **local variables in functions do not retain their values from the previous time the function was called.**




A **static variable** is somewhat similar to a local variable in that it has the same scope as a local variable. However, the lifetime is different. A static variable is not destroyed when the end of the block is reached, but only at the end of the program. As a result, **a static variable can be used to remember things from a previous call of a function.**

A static variable is declared by preceding the variable declaration with the keyword **static**. To declare a static variable of type int:

```
static int variable_name;
```

Unlike a local variable, which has a garbage value upon declaration, a static variable is initialized to zero by default. We can also explicitly specify the value with which to initialize the static variable. In either case, this initialization is done only once, namely the first time the block is entered. It is not repeated when the block is entered again later.

```
static int variable_name = value;
```

Variables cannot be re-declared within the same scope. For example, the program below results in a compilation error.

```
int main() {
    int x;
    int y = 2;
    int x = y + 2;  // x is declared again in the same scope
}
```

A trickier situation occurs when the scope of one variable is a sub-block of that of a different variable with the same name. This results in **variable shadowing**. Consider the example on the next page. The variable `x`, referred to as "outer `x`", is declared within the scope of `main()`. Normally, this variable would also have been accessible within any sub-blocks of its scope; for example, inside the if-statement. This means that the `printf` statement (marked as the "line of interest") would normally have printed this value of `x`. However, another variable `x`, referred to as "inner `x`", was declared within the if-statement. It ends up shadowing the outer `x` within this block. As a result, the line-of-interest `printf` statement uses this inner `x` instead.

A **global variable** is a variable that is declared outside of all blocks. Its lifetime is from the moment of declaration until the end of the program. Similarly, its scope is from its declaration until the last line of the program. The example below illustrates this behavior with global variable `count`.

Global variables are also by default initialized to zero. We could have omitted the "`= 0`" in the declaration of `count`, and the behavior would not have changed.

```
#include <stdio.h>

int count = 0;          // Global variable declaration

void inc() {
   count++;
}

int main() {

    printf("Count = %d   ", count);

    count = 2;
    printf("%d   ", count);

    inc();
    printf("%d   ", count);
}
```

```
Count = 0   2   3
```

| | Local | Static | Global |
|---|---|---|---|
| Declaration | `int x;` Inside a block | `static int x;` Inside a block | `int x;` Outside all blocks |
| Scope | From its declaration to the end of its block | From its declaration to the end of its block | From its declaration to the end of the program |
| Lifetime | While the block executes | From its declaration until the end of the program | From its declaration until the end of the program |
| Initialization | Not initialized | Zero | Zero |

## POINTER VARIABLES

```
int main() {
    // Tying the * to the variable:
    // This is easier to interpret
    int *a, *b;     // a and b are both int pointers
    int *c, d;      // c is an int pointer, but d is an int

    // Tying the * to the type keyword (int in this case):
    // You may be fooled to think e and f are both pointers,
    // but this is incorrect
    int* e, f;      // e is an int pointer, but f is an int
}
```

| Operator name | Symbol | Description |
|---|---|---|
| address | & | Returns the memory address of the variable |
| dereferencing | * | Returns the value found at the memory address that is stored in the variable |

We mentioned earlier that, just as with other variables, newly declared pointer variables without initialization contain garbage values. This is especially dangerous for pointers. If you were to use such un-initialized pointer in a dereferencing operation, you could try to write a value to some random location in memory. This happens in the program below. The variable `ptr` contains a random address, and we write the value 89 to this random address.

```
int main() {
    int *ptr;

    *ptr = 89;
}
```

This could potentially crash your program, or your entire computer, resulting in what is called a "**segmentation fault**" and mess up your data. So be careful with leaving pointers un-initialized. If possible, it is recommended to only declare a pointer once you are ready to assign an address to it (although this is not always practical).

```
#include <stdio.h>

void add(int *ptr) {
    *ptr += 1;
    printf("Inside the function: *ptr = %d", *ptr);
}

int main() {
    int a = 1;
    printf("Value before: a = %d", a);
    add(&a);
    printf("Final Value:  a = %d", a);
}
```

```
Value before: a = 1
Inside the function: *ptr = 2
Value after:  a = 2
```

### 4.3.3 Pointers as Function Return Types

Functions can also have pointers as a return type. This is illustrated in the example below. Note that we chose our variable names in such a way that it is easy to recognize them as pointers. This is purely our own choice. However, it is suggested to adopt a similar convention yourself as it makes it easier to keep track of pointers versus other variables.

```
// This function returns a pointer
int* largest(int *x_ptr, int *y_ptr) {
    if (*x_ptr >= *y_ptr)     // Comparing the content
        return x_ptr;         // Returning the pointer
    else
        return y_ptr;
}

int main() {
    int a = 2, b = 3, *ptr;

    ptr = largest(&a, &b);    // Pointer to the largest of a or b
    *ptr = *ptr * 2;          // Double that one's value
}
```

The example below illustrates the use of `scanf()`. As with `printf()`, the conversion character `%d` refers to an integer[2]. It is important to remember to pass to `scanf()` the addresses of the variables, rather than the variables themselves (hence the use of **`&`**).

```
#include <stdio.h>            // Include the stdio library

int main() {
    int a, b;

    printf("Enter the first number: ");
    scanf("%d", &a);
    printf("Enter the second number: ");
    scanf("%d", &b);
    printf("%d + %d = %d", a, b, a+b);
}
```

```
Enter the first number: 4
Enter the second number: 9
4 + 9 = 13
```

## INTEGER TYPES

| | |
|---|---|
| **`char`** | It uses exactly 8 bits, which is 1 byte. |
| **`short`** | It is implementation specific but guaranteed to be at least 2 bytes (16 bits). |
| **`long`** | It uses at least 4 bytes (32 bits). |
| **`long long`** | It uses at least 8 bytes (64 bits). |

In addition to the minimum sizes mentioned above, the standard also specifies that the following need to be true:

$$\text{sizeof(char)} \leq \text{sizeof(short)} \leq \text{sizeof(int)} \leq \text{sizeof(long)} \leq \text{sizeof(long long)}$$

When you attempt to assign a value that falls outside of the representable range of the type of the variable, **overflow** occurs. The result is that the value that is actually assigned is different from what you intended it to be.

```
#include <stdio.h>

int main() {
    int a = 10000000000000000000;
    printf("%d\n", a);
}
```

```
-1486618624
```

Thus far, we have treated the different types as holding both negative and positive numbers. We refer to this as them being **signed**. As discussed in Appendix C. Bits and Numbers, there are several possible ways in negative numbers can be represented using bits. What they all have in common is that one bit serves as a sign bit.

However, if we knew all our numbers were positive, we could instead use that extra bit to extend our range of numbers. In this case, we are using what is called **unsigned** number representation. This is nothing more than using all bits to represent regular positive binary numbers only.

The keywords **`signed`** and **`unsigned`** can be added in a variable declaration before any of the integer data types we discussed thus far in this chapter, to specify whether we want to use signed or unsigned number representation. By default, all the types (`int`, `short`, `long`, `long long`) are signed, so the `signed` keyword is almost always omitted. The only exception is the `char`

For variables of type `int`, we saw that `%d` is used as the conversion character in `printf()` or `scanf()`. For the other types, the conversion characters are listed in the table below.

| Type | Signed | Unsigned |
|---|---|---|
| char, short, int | %d | %u |
| long | %ld | %lu |
| long long | %lld | %llu |

The example below illustrates the use of the signed and unsigned keywords. Internally data is simply represented by a collection of bits. The signed versus unsigned really just specifies how to interpret those bits. In the `printf` statement, the conversion character `%u` is used to indicate that the data should be interpreted as an unsigned number.

```
#include <stdio.h>

int main() {
   int a;
   signed int b;
   unsigned int c;

   a = -1;
   b = -2;
   c = 3000000000;
   printf("Numbers: %d %d %u \n", a, b, c);
}
```

```
Numbers: -1 -2 3000000000
```

In the C standard[2], fixed-width integers were defined for this reason. They are part of the **stdint** library. This means you need to include following header file:

```
#include <stdint.h>
```

The table below lists the keywords for fixed-width integer types to be used in variable declarations. Mathematical operations can be applied to them the same way as for other integer types.

| Storage size | Signed | Unsigned |
|---|---|---|
| 1 byte | **`int8_t`** | **`uint8_t`** |
| 2 bytes | **`int16_t`** | **`uint16_t`** |
| 4 bytes | **`int32_t`** | **`uint32_t`** |
| 8 bytes | **`int64_t`** | **`uint64_t`** |

## ASCII

```
#include <stdio.h>

int main() {
   char a = 65, b = 'g';
   char c;

   c = b + 1;
   printf("As integers:   %d  %d  %d \n", a, b, c);
   printf("As characters: %c  %c  %c \n", a, b, c);
}
```

```
As integers:   65  103  104
As characters: A  g  h
```

**ASCII table**

| Decimal | Hex | Char | Decimal | Hex | Char | Decimal | Hex | Char | Decimal | Hex | Char |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | [NULL] | 32 | 20 | [SPACE] | 64 | 40 | @ | 96 | 60 | ` |
| 1 | 1 | [START OF HEADING] | 33 | 21 | ! | 65 | 41 | A | 97 | 61 | a |
| 2 | 2 | [START OF TEXT] | 34 | 22 | " | 66 | 42 | B | 98 | 62 | b |
| 3 | 3 | [END OF TEXT] | 35 | 23 | # | 67 | 43 | C | 99 | 63 | c |
| 4 | 4 | [END OF TRANSMISSION] | 36 | 24 | $ | 68 | 44 | D | 100 | 64 | d |
| 5 | 5 | [ENQUIRY] | 37 | 25 | % | 69 | 45 | E | 101 | 65 | e |
| 6 | 6 | [ACKNOWLEDGE] | 38 | 26 | & | 70 | 46 | F | 102 | 66 | f |
| 7 | 7 | [BELL] | 39 | 27 | ' | 71 | 47 | G | 103 | 67 | g |
| 8 | 8 | [BACKSPACE] | 40 | 28 | ( | 72 | 48 | H | 104 | 68 | h |
| 9 | 9 | [HORIZONTAL TAB] | 41 | 29 | ) | 73 | 49 | I | 105 | 69 | i |
| 10 | A | [LINE FEED] | 42 | 2A | * | 74 | 4A | J | 106 | 6A | j |
| 11 | B | [VERTICAL TAB] | 43 | 2B | + | 75 | 4B | K | 107 | 6B | k |
| 12 | C | [FORM FEED] | 44 | 2C | , | 76 | 4C | L | 108 | 6C | l |
| 13 | D | [CARRIAGE RETURN] | 45 | 2D | - | 77 | 4D | M | 109 | 6D | m |
| 14 | E | [SHIFT OUT] | 46 | 2E | . | 78 | 4E | N | 110 | 6E | n |
| 15 | F | [SHIFT IN] | 47 | 2F | / | 79 | 4F | O | 111 | 6F | o |
| 16 | 10 | [DATA LINK ESCAPE] | 48 | 30 | 0 | 80 | 50 | P | 112 | 70 | p |
| 17 | 11 | [DEVICE CONTROL 1] | 49 | 31 | 1 | 81 | 51 | Q | 113 | 71 | q |
| 18 | 12 | [DEVICE CONTROL 2] | 50 | 32 | 2 | 82 | 52 | R | 114 | 72 | r |
| 19 | 13 | [DEVICE CONTROL 3] | 51 | 33 | 3 | 83 | 53 | S | 115 | 73 | s |
| 20 | 14 | [DEVICE CONTROL 4] | 52 | 34 | 4 | 84 | 54 | T | 116 | 74 | t |
| 21 | 15 | [NEGATIVE ACKNOWLEDGE] | 53 | 35 | 5 | 85 | 55 | U | 117 | 75 | u |
| 22 | 16 | [SYNCHRONOUS IDLE] | 54 | 36 | 6 | 86 | 56 | V | 118 | 76 | v |
| 23 | 17 | [ENG OF TRANS. BLOCK] | 55 | 37 | 7 | 87 | 57 | W | 119 | 77 | w |
| 24 | 18 | [CANCEL] | 56 | 38 | 8 | 88 | 58 | X | 120 | 78 | x |
| 25 | 19 | [END OF MEDIUM] | 57 | 39 | 9 | 89 | 59 | Y | 121 | 79 | y |
| 26 | 1A | [SUBSTITUTE] | 58 | 3A | : | 90 | 5A | Z | 122 | 7A | z |
| 27 | 1B | [ESCAPE] | 59 | 3B | ; | 91 | 5B | [ | 123 | 7B | { |
| 28 | 1C | [FILE SEPARATOR] | 60 | 3C | < | 92 | 5C | \ | 124 | 7C | \| |
| 29 | 1D | [GROUP SEPARATOR] | 61 | 3D | = | 93 | 5D | ] | 125 | 7D | } |
| 30 | 1E | [RECORD SEPARATOR] | 62 | 3E | > | 94 | 5E | ^ | 126 | 7E | ~ |
| 31 | 1F | [UNIT SEPARATOR] | 63 | 3F | ? | 95 | 5F | _ | 127 | 7F | [DEL] |

## FLOATING POINT NUMBERS

**`float`** This type uses 4 bytes to represent floating point numbers. It is referred to as being single precision.

**`double`** This type, the name of which stands for double precision, uses 8 bytes. In practice, floating point numbers are typically defined as `double`, rather than `float`. For even higher precision, there exists a `long double` as well, with a storage size of 10 bytes.

The mathematical operations that can be applied to floating point data are the ones you would expect and are familiar with. Note, however, that floating point types do not support the modulo operator `%` that integers did.

| Operator | |
|---|---|
| Addition | + |
| Subtraction | - |

| Operator | |
|---|---|
| Multiplication | * |
| Division | / |

| Type | Decimal notation | Scientific notation | Simplified notation |
|---|---|---|---|
| float (scanf/printf), double (printf) | %f | %e | %g |
| double (scanf/printf) | %lf | %le | %lg |

```
#include <stdio.h>

int main() {
   double a = 133.422;
   double b = 32.5e-4;

   printf("a = %lf  %le  %lg \n", a, a, a);
   printf("b = %lf  %le  %lg \n", b, b, b);

   printf("a = %1.2f \n", a);
}
```

```
a = 133.422000  1.334220e+02  133.422
b = 0.003250  3.250000e-03  0.00325
a = 133.42
```

When the operands of an expression are of **different types**, however, all the operands get **promoted** to the highest type of any of the operands, according the following ranking:

**char → short → int → long → float → double**

For example, when adding an `int` and a `float`, the `int` will first be converted to type `float` and the result of the addition will be of type `float` as well. The tables below illustrate this further.

| Declarations |
|---|
| char c = 2;<br>short s = -15;<br>int i = 2;<br>long j = 9;<br>float f = 1.5;<br>double d = 25; |

| Expression | Resulting type | Resulting value |
|---|---|---|
| c + c | int | 4 |
| c + s | int | -13 |
| c - s/i | int | 9 |
| j * 2.0 - i | double | 16.0 |
| j + 3 | long | 12 |
| c + 5.0 | double | 7.0 |
| f * 7 - i | float | 8.5 |
| d + c | double | 27.0 |
| f * d - j | double | 28.5 |
| j / i * 1.5 | double | 6.0 |

```
#include <stdio.h>

int main() {

   double y;
   int x, z = 5;

   x = z * 1.1; // Expression evaluates to 5.5 (type double)
                // This is converted to 5 (type int)
   y = z / 4;   // Expression valuates to 1 (type int)
                // This is converted to 1.0 (type double)

   printf("x: %d \n", x);
   printf("y: %1.2f \n", y);
}
```

```
x: 5
y: 1.00
```

## 5.5.3 Casting

The type promotions and conversions discussed in the previous two sections are performed automatically in C. However, sometimes you may want to explicitly force a change of type. This can be done using the **type casting operator**. This operator takes the form of the type to be cast to in parentheses, followed by the operand. It is a unary operator. For example:

| | |
|---|---|
| `(int)a` | Type cast `a` to an `int` |
| `(double)b` | Type cast `b` to a `double` |

```
#include <stdio.h>

int main() {
   int i = 3, j = 2;
   double x, y = 3.14;

   x = i/j;                // Integer division: i/j is 1
   printf("x1: %1.2f \n",x);

   x = i/(double)j;        // Typecast j; floating point division
   printf("x2: %1.2f \n",x);

   x = (double)i/j;        // Typecast i; floating point division
   printf("x3: %1.2f \n",x);

   x = (double)(i/j);      // Typecast i/j; integer division
   printf("x4: %1.2f \n",x);

   x = (int)y;             // Result is int (value 3)
   printf("x5: %1.2f \n",x);
}
```

```
X1: 1.00
X2: 1.50
X3: 1.50
X4: 1.00
X5: 3.00
```

## POINTERS TO POINTERS

```
#include <stdio.h>

int main() {
   int a = 1;
   int *ptr1;
   int **ptr2;
   int ***ptr3;
```

```
   ptr1 = &a;       // address of int
   ptr2 = &ptr1;    // address of pointer to int
   ptr3 = &ptr2;    // address of pointer to pointer to int

   printf("%lu %lu %lu \n", ptr1, ptr2, ptr3);
   printf("%lu %lu %d \n", *ptr3, **ptr3, ***ptr3);
   printf("%d %d %d", *ptr1, **ptr2, ***ptr3);
}
```

```
11056  11060  11068
11060  11056  1
1  1  1
```

# 5.8 Random Numbers

C also provides support for generating random numbers, as part of the **stdlib** library. To use this library, you must add the line `#include <stdlib.h>` at the top of your program (as with other libraries). The example below illustrates the use of random numbers in a simple program.

```
#include <stdio.h>      // Library needed for printf()
#include <stdlib.h>     // Library needed for rand(), srand()
#include <time.h>       // Library needed for time()

int main() {
  int i, val;

  srand(time(NULL));      // Set the current time as the seed

  for (i = 0; i < 4; i++) {
     val =  rand();       // Generate a (pseudo) random value
                          // between 0 and RAND_MAX
     printf(" %d ", val);
  }
}
```

# 6.1 Array Basics

## 6.1.1 Declaring and Accessing Arrays

We will introduce the basic concepts of arrays by means of the example below.

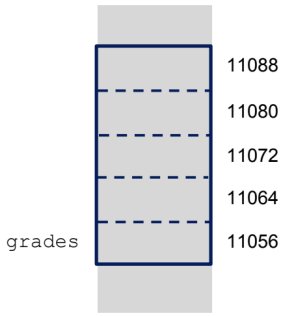
```
#include <stdio.h>

int main() {
    int i = -1;
    double grades[5];            // Array declaration

    grades[0] = 37.5;
    grades[1] = 44.2;
    grades[2] = grades[0] + grades[i+2];

    printf("%1.1f %1.1f %1.1f", grades[0], grades[1], grades[2]);
}
```

```
37.5  44.2  81.7
```

**Declaration:** The syntax for an array declaration is as follows: the base type, followed by the variable name, followed by a number in square brackets that gives the size of the array. The size of an array is the number of elements of the base type it can store. In our example, the first line in main() declares an array variable `grades`, which can store 5 elements of type `double`.

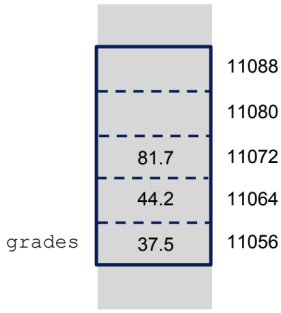
```
double grades[5];
```

The figure on the right shows the corresponding memory diagram representation. A **contiguous** block of memory is reserved, large enough to store 5 doubles. Note that the elements in the array always occupy consecutive blocks in memory.



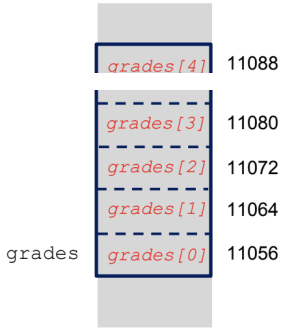

**Indexing:** Accessing the individual elements inside an array is referred to as indexing. The diagram on the right shows the memory contents at the end of the program. The third, fourth and fifth line in `main()` illustrate array indexing, both in terms of assigning data to array elements and reading data from array elements. To access a specific element in the array, we use the array name followed by the element number in square brackets. For example, `grades[0]` refers to the first element in the array.

```
grades[0] = 37.5;
```




Note that each array element is a variable of the array base type (in this case, a double). We can use the array element anywhere we have used variables of primitive types in the past. Also, everything we know about type casting, conversion, etc. is still valid here as well.

It is important to point out that C uses what is known as **zero-indexing**. This means that the first array element has an index 0, the second has index 1, and so on. This is illustrated in the diagram on the right (*the labels in red are not the content in memory; they show the index numbers*). The index into an array must be an integer or an integer expression (i.e., evaluate to an integer). This is illustrated in the fifth line of `main()` , where `i+1` is used as the index. This evaluates to the value 1, and thus accesses `grades[1]`.




Note that C does not check for **out-of-bound array indexing**. This means that it does not check whether you are trying to access an array element that is not in the array. In our example, `grades[5]` would be the 6th element of the array `grades`. However, the array only has a size of 5, and this index is therefore out of bounds. What would happen in this case? If you read from an index that is out of bounds, C will look at the memory that would have been part of the array had it been large enough. Typically, this will result in a non-sensical value. However, if you were to write to an out-of-bounds array element, you may be trying to modify parts of the memory you shouldn't. This will likely cause the program to crash (i.e., cause a run-time error), more specifically result in a **segmentation fault**. It is good to be aware that errors resulting from mistakes in array indexing are unfortunately common and often hard to debug.

Finally, it is worth noting that all elements in an array are always of the same type. In our example, this was the type `double`, but you can create arrays of any type. This includes primitive types, but also any other data structures we will cover in later chapters (such as structs).

```
#include <stdio.h>

int main() {
    int a[4] = {1, 4, 5, 8};
    int b[4] = {6, 7};
    int c[4] = {};

    printf("a = %d %d %d %d \n", a[0], a[1], a[2], a[3]);
    printf("b = %d %d %d %d \n", b[0], b[1], b[2], b[3]);
    printf("c = %d %d %d %d \n", c[0], c[1], c[2], c[3]);
}
```

```
a = 1  4  5  8
b = 6  7  0  0
c = 0  0  0  0
```

```
#include <stdio.h>

int main() {
    int a[4] = {1, 4, 5, 8};
    int b[]  = {1, 4, 5, 8};

    printf("a = %d %d %d %d \n", a[0], a[1], a[2], a[3]);
    printf("b = %d %d %d %d \n", b[0], b[1], b[2], b[3]);
}
```

```
a = 1  4  5  8
b = 1  4  5  8
```

In our examples thus far, we have used arrays as local variables. However, as with other data types we have seen, they can also be used as global or static variables. In both these cases, the array elements are all automatically initialized as zeros.

```
#include <stdio.h>

// Global array variable (initialized to all 0)
int x[10];

void process() {
    // Local array variable (uninitialized)
    int y[10];

    // Static array variable (initialized to all 0)
    static int z[10];
}

int main() {
    process();
}
```

## 6.1.3 Passing Arrays to Functions - Basics

The most straightforward way to interface arrays with functions is to pass individual array elements as function arguments, as illustrated in the example below. Array elements, obtained with the [ ] operator, behave just as any other variable of the appropriate type (in the example, as a double). As always, they are **passed by value**, which means that they are not changed by the function. So even though `a[0]` gets mapped to `x`, its value in `main()` does not change. This is exactly as you would expect based on what we have seen before.

```
#include <stdio.h>

double process(double x, double y) {
    x = x + 1;
    return x + y;
}

int main() {
    double a[3] = {4.1, 2.5, 3.3};
    double b;

    b = process( a[0] , a[1] );
    printf("%1.1f  %1.1f", b, a[0]);
}
```

```
7.6    4.1
```

```c
#include <stdio.h>

void modify(double x[], int length) {
    int i;

    for (i=0; i<length; i++)
       x[i] = 7.0;
}

int main() {

    double a[3] = {4.1, 2.5, 3.3};

    modify( a , 3 );
    printf("%1.1f  %1.1f  %1.1f", a[0], a[1], a[2]);
}
```

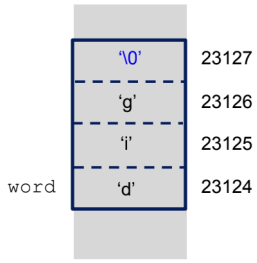
```
7.0   7.0   7.0
```

## 6.2 Strings

You may remember that we have encountered the term "string" before, when we talked about the `printf()` function and its format string. There we understood a "string" to essentially be a sequence of characters.

In C, a string is formally defined as an **array of characters, terminated by the null termination character '\0'**. What does this mean, and what is this null character?

We will explain this using the example below. On the first line of `main()`, an array is declared, consisting of 4 elements of type `char`. Remember, the data type `char` holds 1-byte integer values, which can encode characters as specified by the ASCII table (see Chapter 5). In this case, we initialized the array `word` with 4 characters: the letters 'd', 'i' and 'g' and the special character '\0'.




```c
#include <stdio.h>

int main() {
    char word[4] = {'d', 'i', 'g','\0'};

    word[1] = 'o';
    int i = 0;
    while (word[i] != '\0') {
       printf("%c", word[i]);
       i++;
    }
}
```

```
dog
```

Now let's go back to actual strings (i.e., character arrays with the null termination character). When printing all the characters in a string, we can use a while loop as we did in the example on the previous page. However, C also provides a convenient shortcut with `printf()`: the conversion character **`%s`**. Its use is illustrated below. This `%s` conversion character essentially behaves as a shorthand for the while-loop from the earlier example: it prints each element of the array as a character, until the terminal character is encountered. Note that the second argument to `printf()` in this case is the entire array (i.e., `word`, in our example).

```c
#include <stdio.h>

int main() {
    char word[4] = {'d', 'i', 'g', '\0'};

    word[1] = 'o';
    printf("%s", word);
}
```

```
dog
```

Since `printf()` with %s only prints until the termination character is encountered, any subsequent characters in the array are ignored, as illustrated below.

```c
#include <stdio.h>

int main() {
    char word[6] = {'m', 'a', 'p', '\0', 'l', 'e'};
    printf("%s", word);
}
```

```
map
```

In our examples thus far, we have initialized a string by specifying each element (including the termination character) within { }. However, a more common approach is to instead use a **string constant**. A string constant is a set of symbols within double quotes "", such as "dig" or "C is a programming language.", which implicitly adds a null termination character at the end.

A string constant can be used to initialize a string, as shown in the example below. Note that it automatically includes the '\0' at the end. This means that the array `word1` in the example is of length 4, containing the characters 'd','i','g','\0'. It is therefore important not to forget that the string constant always represents one character more than the ones actually visible. If we were to specify the length of `word1` explicitly instead of leaving [ ] empty, we would have to allocate a length of at least 4. The special case of "" is called the null-string. In the example below, `word2` is initialized with this null-string; it is an array of size 1 containing only the null character ('\0').

```c
#include <stdio.h>

int main() {
    char word1[] = "dig";
    char word2[] = "";

    printf("word1: %s  word2: %s", word1, word2);
}
```

```
Word1: dig   word2:
```

To further illustrate this, consider the three programs below. In each case, adding one to the pointer results in it pointing to the next integer, char or double respectively. This means that `ptr1` contains an address that is 4 more than the one it stored originally, as we saw above. However, for `ptr2`, that is only 1 more, as a char only consists of 1 byte. Similarly, for `ptr3`, that is 8 more (assuming doubles are 8 bytes on our system).

```c
int main() {
  int a[2] = {1,2};
  int *ptr1 = &a[0];

  ptr1 += 1;
}
```

```c
int main() {
  char a[2] = {1,2};
  char *ptr2 = &a[0];

  ptr2 += 1;
}
```

```c
int main() {
  double a[2] = {1,2};
  double *ptr3 = &a[0];

  ptr3 += 1;
}
```

### 6.3.3 Allowed Pointer Arithmetic

Pointer arithmetic only allows a limited set of operations. The reason is that only certain operations on addresses make sense.

1. **Adding or subtracting an integer constant or expression**

   This is a generalization of what we discussed earlier. Adding a value x to a pointer causes it to increase by x elements of the base data type. Some examples are shown below (we are assuming that `a` is a variable of an integer data type):

   ```c
   ptr++;          ptr1 = ptr2 + (a-4);          ptr3 -= 89;
   ```

2. **Subtracting two pointers of the same type**

   The result of this operation is the distance in memory between the two pointers, expressed in number of elements of the data type. In the example program below, variable `b` ends up having a value of 2 at the end: the two pointers contain addresses that are two integers apart. Note that the result of pointer subtraction can be positive or negative. Also, it is important to emphasize that pointer subtraction is only allowed if the pointers are of the same type.

   ```c
   int main() {
       int a[3] = {3, 42, -9};
       int b;
       int *ptr1 = &a[0];
       int *ptr2 = &a[2];

       b = ptr2 - ptr1;
   }
   ```

### 6.3.4 Combining Pointer Arithmetic and Dereferencing

Now let's combine dereferencing with pointer arithmetic. This will allow us to access the elements in an array. Consider the example below. In the first `printf()` statement, the pointer refers to the first element of the array and dereferencing it will yield that first element. In the second `printf()`, `(ptr+1)` points to an address that is one integer further, or essentially the next element in the integer array, and so on.

```c
#include <stdio.h>

int main() {
    int a[3] = {423, 12, -780};
    int *ptr;

    ptr = &a[0];

    printf("%d \t", *ptr     );
    printf("%d \t", *(ptr+1) );
    printf("%d \t", *(ptr+2) );
}
```

```
423     12      -780
```

This combination of pointer arithmetic and dereferencing is so common in C (and we will see why that is the case in the next section) that a short-hand notation is used for it: the **[ ] operator**. The two notations below are equivalent:

| `*(ptr+n)` | `ptr[n]` |
|---|---|

More specifically, in almost all cases (and we will discuss the exceptions in the next section), **the array name gets to automatically converted to a pointer to the first element in the array**.

Consider the example below. On the last line of code, the array `a` is assigned to pointer `ptr`. Note that these variables are of very different types: `a` is an array and `ptr` is a pointer. However, in this expression, array name `a` gets converted automatically to a pointer to its first element (i.e., something of type pointer-to-int), which is then assigned to `ptr`. We could, in fact, have replaced this last line of code, with the following, for an identical result:

```
ptr = &a[0];
```

```c
int main() {
    int a[5] = {4,2,9,1,3};
    int *ptr;

    ptr = a;
}
```

| | |
|---|---|
| (1) Assigning an array to a pointer | `int *ptr = a;` |
| (2) Performing pointer arithmetic on an array | `int x = *(a+1);` |
| (3) Accessing the array elements | `int x = a[1];` |
| (4) Passing an array to a function | See section 6.4.3 |

1. An array declaration reserves space in memory for n values of a certain type. By contrast, a pointer can only hold an address.
2. Once declared, arrays have a fixed place in memory. As a result, assigning to array names is not allowed. Consider the examples below:

**Allowed: assigning to pointers**

```c
int main() {
    int a[3] = {4,2,9};
    int *ptr1, *ptr2;

    ptr1 = a;
    ptr2 = a+1;

    ptr2 = ptr1;    // CORRECT
    ptr1++;         // CORRECT
}
```

**Not allowed: assigning to arrays**

```c
int main() {
    int a[3] = {4,2,9};
    int b[3] = {5,7,8};



    b = a;          // ERROR
    a++;            // ERROR
}
```

6.1.3. However, both these notations are completely equivalent when used in a function specification:

```
double *x       double x[]
```

The second option with the empty [ ] is reminiscent of an array declaration. However, it is really just an alternative way of defining the pointer `x`. It does not define an actual array. It is merely a notational analogue for a pointer when used in the parameter list of a function definition.

```c
#include <stdio.h>

void modify(double *x, int length) {
    int i;

    for (i=0; i<length; i++)
       x[i] = 7.0;
}

int main() {
    double a[3] = {4.1, 2.5, 3.3};

    modify( a , 3 );

    printf("%1.1f  %1.1f  %1.1f", a[0], a[1], a[2]);
}
```

```
7.0   7.0   7.0
```

```c
// THIS PROGRAM IS WRONG; IT DOES NOT COMPILE
int main() {
    char word1[] = "Hello";
    char word2[10];

    word2 = word1;            // ERROR
}
```

```c
#include <stdio.h>

int main() {

    char word1[] = "Hello";
    char word2[] = "Hello";

    printf("%d \n", word1 == word2 );    // Compares the address
    printf("%d \n", word1 == word1 );    // Compares the address
}
```

```
0
1
```

Finally, we also want to discuss a few things about how strings interact with the library functions `printf()` and `scanf()`. As was mentioned before, using the **`%s`** conversion character with `printf()` enables you to print the string until the null termination character. The argument that is passed to the function is the string array name (in the example, the array `text`).

```c
#include <stdio.h>

int main() {
    char text[100];

    scanf("%s", text);
    printf("%s", text);
}
```

```
This is a test
This
```

## 6.5.1 2D Array Basics

Consider the example shown below. The first line in `main()` declares a 2D array of integers of dimensions 2-by-3. The syntax is similar to what we saw for regular (1D) arrays, and as was the case there, the element values are uninitialized (garbage values). The subsequent lines illustrate how array elements can be accessed, using the familiar square bracket notation.

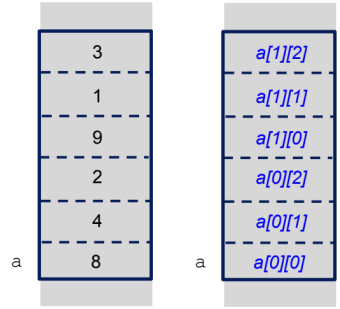
```c
int main() {
    int a[2][3];              // Declaration

    a[0][0] = 8;              // Accessing array elements
    a[0][1] = 4;
    a[0][2] = 2;
    a[1][0] = 9;
    a[1][1] = 1;
    a[1][2] = 3;
}
```

| | | |
|---|---|---|
| 8 | 4 | 2 |
| 9 | 1 | 3 |

| | | |
|---|---|---|
| *a[0][0]* | *a[0][1]* | *a[0][2]* |
| *a[1][0]* | *a[1][1]* | *a[1][2]* |

The 2D array can be thought of as the 2D data structure shown on the previous page. Often, people visualize the first dimension as the rows and the second as the columns. This corresponds with how matrices are indexed in math. Note that indexing again starts at 0 (i.e., **zero indexing**).

In principle, this visualization is somewhat arbitrary. What is fixed, however, is the fact that there is a first and second dimension to the array, and how the data is actually stored in memory. This is shown on the right. If visualized as a matrix, it means elements are stored sequentially row-by-row. All elements occupy adjacent spots in memory.




As with regular arrays, any missing elements in the initialization default to zeros. In the table below, the right column provides the equivalent initialization for the abbreviated one on the left.

| | | |
|---|---|---|
| `int a[2][3] = {{8},{9,1}};` | → | `int a[2][3] = {{8,0,0},{9,1,0}};` |
| `int a[2][3] = {{8,4}};` | → | `int a[2][3] = {{8,4,0},{0,0,0}};` |
| `int a[2][3] = {{}};` | → | `int a[2][3] = {{0,0,0},{0,0,0}};` |

Also, you can provide the initialization values as a one-dimensional list. In this case, the elements essentially directly map to how they are stored in memory as was shown in the memory diagram earlier. Missing elements are again replaced by zeroes.

| | | |
|---|---|---|
| `int a[2][3] = {8,4,2,9,1,3};` | → | `int a[2][3] = {{8,4,2},{9,1,3}};` |
| `int a[2][3] = {8,9,1,3};` | → | `int a[2][3] = {{8,9,1},{3,0,0}};` |
| `int a[2][3] = {};` | → | `int a[2][3] = {{0,0,0},{0,0,0}};` |

| | | |
|---|---|---|
| `int a[][3]  = {8,4,2,9,1,3};` | → | `int a[2][3] = {{8,4,2},{9,1,3}};` |
| `int a[][2]  = {8,4,2,9,1,3};` | → | `int a[3][2] = {{8,4},{2,9},{1,3}};` |

The most important things to remember are:

1. All except the first dimension need to be specified when a multidimensional array is used as a function parameter.
2. Any modifications to the array in the function change the original array, as this act as a call by reference.

```
#define ROWS     3
#define COLUMNS  2

void invert(int x[][COLUMNS]) {
    int i, j;
    for (i = 0; i < ROWS; i++)
       for (j = 0; j < COLUMNS; j++)
           x[i][j] = -x[i][j];
}

int main() {
    int a[ROWS][COLUMNS] = {{4,8},{2,3},{4,7}};
    invert(a);
}
```

```
void invert(int *x, int size) {
    int i;
    for (i = 0; i < size; i++)
           x[i] = -x[i];
}

int main() {
    int a[3][2] = {{4,8},{2,3},{4,7}};
    invert(&a[0][0], 6);
}
```

The code below is an example of 3D arrays. For the array declarations, missing elements are again set to zero (see array `a`). If the array dimensions are not fully specified, they can be derived from the number of elements in the initialization (see array `b`). However, only the first dimension can be left unspecified. Similarly, in the function parameter list, the first dimension can be omitted, but all others need to be specified. Functions are again called by reference.

```
#define DIM1     3
#define DIM2     2
#define DIM3     2

void half(int x[][DIM2][DIM3]) {
    int i, j, k;

    for (i = 0; i < DIM1; i++)
       for (j = 0; j < DIM2; j++)
           for (k = 0; k < DIM3; k++)
               x[i][j][k] /= 2;
}

int main() {
    int a[DIM1][DIM2][DIM3] = {{7,8},{5}};
    int b[][DIM2][DIM3] = {9,8,7,6,5,4,3,2,1,0,-1,-1};

    half(a);
    half(b)
}
```

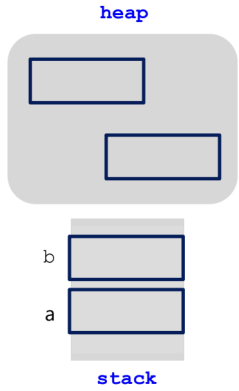
## MEMORY ORGANIZATION

Whenever we have declared local variables in our program thus far, either as simple variables or as arrays, we were allocated memory on the **stack**. On the stack, memory is organized in a very structured way: each time a new variable is declared, it is given the next memory location. Another important property of the stack is that at the end of a variable's lifetime, its memory is released automatically.

The heap memory segment, on the other hand, is organized very differently from the stack. It is basically treated as a block of unorganized memory. When a program requests a chunk of memory on the heap, it is allocated a contiguous block, but this block can be anywhere on the heap, and is not necessarily adjacent to the previous block that was allocated.

Heap memory is useful when dealing with large amounts of data. Also, the program can decide at run-time how much memory on the heap it needs[1]. The heap is therefore also useful when there is significant benefit from only requesting the amount of data that is needed at run-time. Note that because the amount of memory needed is decided at run-time, the act of requesting memory on the heap is known as **dynamic memory allocation**.

As mentioned before, the stack and the heap are simply two logical segments in the same physical memory. We can show this explicitly in the memory representation, as is illustrated in Appendix D. Memory Organization. However, in the remainder of this chapter, we will use the more simplified, abstract memory representation shown on the right. It emphasizes the different structures of the stack and heap (structured versus unstructured). In reality, of course, memory on the heap is not really scattered in 2D; we merely use this representation to emphasize the unstructured nature.




## 7.2 Dynamic Memory Allocation

The functions that enable you to request and manipulate memory on the heap are available through the **stdlib** library. Therefore, for all the dynamic memory allocation functions we will discuss next, this library needs to be included in your program:

```
#include <stdlib.h>
```

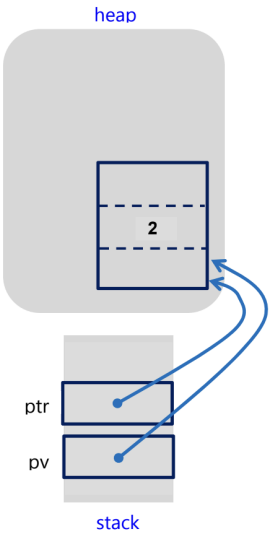
### 7.2.1 malloc()

The `malloc()` function below requests a contiguous block of **n** bytes of memory on the heap. If successful (i.e., a block of that size can be found), the function returns a pointer to that memory.

```
void* malloc(int n)
```

Since `ptr` has a base type (`int` in this case), it can be used with all the pointer operations we have seen before: dereferencing, pointer arithmetic, etc.

```
#include <stdlib.h>

int main() {
   void *pv;
   pv = malloc(12);
   int  *ptr;
   ptr = (int*)pv;    // pointer casting
   *(ptr+1) = 2;
}
```




In fact, we will always combine dynamic memory allocation with pointer casting to specify how (i.e., as what data type) the newly allocated memory needs to be interpreted. As such, you will typically find the code written as shown below, with the two operations combined in the same line of code and foregoing the declaration of a separate void pointer.

```
#include <stdlib.h>

int main() {
   int *ptr;
   ptr = (int*)malloc(3 * sizeof(int));
   *(ptr+1) = 2;
}
```

### 7.2.2 calloc()

The `calloc()` function is an alternative for `malloc()`. It also allocates a contiguous block of memory on the heap and returns a void pointer to this memory (if the allocation is successful). However, the interface is slightly different: you need to specify the number of elements and the size of the elements as two parameters, rather than the product of the two as a single parameter as we did for `malloc()`.

```
void *calloc(int num_elements, int size_element)
```

As with `malloc()`, you would typically use `sizeof()` to determine the size of the element. Also, the void pointer needs to be cast before any data operators are possible on the newly allocated memory. The example below illustrates the use of `calloc()`.

```
#include <stdlib.h>

int main() {
   int *ptr;
   ptr = (int*)calloc(3, sizeof(int));
   *(ptr+1) = 2;
}
```

Unlike `malloc()`, however, `calloc()` **initializes the memory to zero**. This is the main difference between the two functions. As a result, `calloc()` is slightly slower, but not by much.

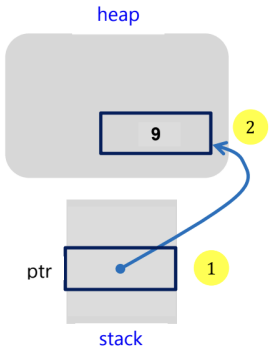
### 7.2.4 Releasing Memory

As mentioned earlier, memory allocated on the heap, which we learned we can do using `malloc()` or `calloc()`, needs to be released explicitly. Contrast this to variables on the stack. When discussing scope and lifetime, we saw that these variables are "destroyed"[3] automatically at the end of their lifetime. This is not the case of data allocated on the heap. Data there will persist until the user explicitly relinquishes it. To release the memory blocks we requested using `malloc()` and `calloc()`, we need to call the `free()` function.

```
void free(void *ptr)
```

The parameter `ptr` is a pointer to the start of the memory block (otherwise an error will occur). Even though it is defined as a void pointer, it accepts any pointer type. If a NULL pointer is passed to `free()`, it would simply do nothing. The function `free()` does not need to know the size of the memory block.

```
#include <stdlib.h>

int main() {
   I   int *ptr;                          1
       void *p = malloc(sizeof(int));     2

   II  ptr = (int*)p;                     1
       *ptr = 9;                          2

   III // automatic                       1
       free(ptr);                         2
}
```




The lines of code that are associated with each of those two pieces are labeled as **1** (the pointer variable) and **2** (the data it points to) respectively. For each piece, there are three types of actions:

I. **The memory is allocated**. This involves (1) the declaration of a pointer variable (on the stack) and (2) the dynamic memory allocation (on the heap) respectively. Both these actions create space in memory to hold data.
II. **The memory is assigned**. This means that the data is stored in the space that was created in step I. Specifically, (1) the pointer on the stack is given the address of the memory block on the heap, and (2) dereferencing is used to assign data on the heap.
III. **The memory is de-allocated**. The memory is given back to be reused by the system. (1) This happens automatically for local variables on the stack. (2) For dynamic memory, this must be done explicitly using `free()`.

## 8.1 Typedef

The **typedef** construct is used to create a simpler synonym for another, often more complex, type definition. The basic syntax is shown below. It consists of the keyword `typedef` followed by the existing type and the new name that we want to define for this type. Note that the existing type can be any type: a primitive type, but also any derived type or data structure.

```
typedef <existing_type> <new_name>;
```

The scope of typedef is similar to that of variables, with a local and a global scope, as demonstrated below. The new types `feet` and `inches` can be used globally, while `euro` is restricted to the function `test()`.

```
#include <stdio.h>

typedef int feet, inches;

feet test() {
   typedef double euro;
   euro x = 0.25, y = 9;
   feet z = 6;
   return z;
}

int main() {
   feet a;
   inches b = 1;
   a = test();
   printf("%dft%d \n",a,b);
}
```

```
6ft1
```

**Structs** are data structures that bear some resemblance to arrays in that they group together multiple elements. However, where for arrays these elements all need to be of the same type, for structs the elements can be of different types. Also, as we will see later, there are other fundamental differences between the two types, such as how they behave in function calls.

Before you can declare variables of a struct data structure, you first need to define the data structure itself, as shown below. This creates a struct type, gives the new struct type a name and specifies the elements that make up the struct. These elements are called "**fields**" and each field is specified by its name and its type.

```
struct <struct_name> {
     type1 <field1_name>;
     type2 <field2_name>;
};
```

As an example, the code below defines a new struct type, called `grid`, with two fields: `symb` (of type `char`) and `num` (of type `int`). A struct can have any number of fields. Also, the field can be of any type: a primitive type, but also any derived type (pointer, array, etc.) or data structure (struct, enum, etc.).

```
struct grid {
   char symb;
   int num;
};
```

We declare a variable of a struct type are follows. Note that we need to include the `struct` keyword.

```
struct <struct_name> <variable_name>;
```

Once a variable of type struct is declared, we can use it in our code. The **. operator** ("dot-operator") allows us to access a specific field in a variable of type struct. This operator has the highest order of precedence (the same as parentheses and [ ]). An updated precedence table is provided at the end of the chapter.

```
<variable_name>.<field_name>
```

```
#include <stdio.h>

struct grid {
   char symb;
   int num;
};

int main() {
   struct grid a, b;
   struct grid c = {'k',3};

   a.num = 2;
   b = a;
   a.symb = 'r';
   b.symb = 'v';

   printf("%c %d \t",a.symb,a.num);
   printf("%c %d \t",b.symb,b.num);
   printf("%c %d \n",c.symb,c.num);
}
```

```
r  2        v  2        k  3
```

## 8.2.2 Structs and Typedef

Often, typedef is used in conjunction with structs to simplify the notation. First, consider the way we have used structs thus far (without typedef) as shown below. Note that variables of our struct type need to be declared as `struct point`, followed by the variable names. To aid our upcoming discussion, we have highlighted the struct definition in yellow.

```
#include <stdio.h>

struct point {
     double x;
     double y;
 };

int main() {
    struct point a, b;
}
```

We can now use typedef to create a synonym for our struct type. Remember, the syntax for typedef was the keyword `typedef`, followed by the existing type and the new name for it:

```
typedef <existing_type> <new_name>;
```

In this case, the existing type is the entire struct definition, which was marked in yellow in the earlier example. The result is illustrated below, where typedef is used to create a new name `coord` for this struct. It is now possible to declare variables two ways:

1. By using the struct type directly, i.e., `struct` followed by the struct name (as illustrated with variable `a`).
2. By using the new type created with typedef (as illustrated with variable `b`).

```
#include <stdio.h>

typedef struct {
            double x;
            double y;
        }               coord;

int main() {
    coord a, b;
}
```

```
#include <stdio.h>

typedef struct {
    double x;
    double y;
} coord;

int main() {
    coord a = {1.5, 5.2};
    coord *b;

    b = &a;

    (*b).x += 1;
    b->x   += 1;

    printf("%1.1f   %1.1f \n",a.x, b->x);
}
```

```
3.5   3.5
```

## Structs are passed by value:

```
#include <stdio.h>

typedef struct {
    double x;
    double y;
} coord;

coord process(coord k) {

    k.x = k.y;

    return k;
}

int main() {
    coord a = {1.5, 5.2}, b;

    b = process(a);

    printf("(%1.1f, %1.1f) \n", a.x, a.y);
    printf("(%1.1f, %1.1f) \n", b.x, b.y);
}
```

```
(1.5, 5.2)
(5.2, 5.2)
```

## Struct passed by pointer:

```
#include <stdio.h>

typedef struct {
    double x;
    double y;
} coord;

coord* process(coord *k) {

    k->x = k->y;

    return k;
}

int main() {
    coord a = {1.5, 5.2};
    coord *b;

    b = process(&a);

    printf("(%1.1f, %1.1f) \n", a.x, a.y);
    printf("(%1.1f, %1.1f) \n", b->x, b->y);
}
```

```
(5.2, 5.2)
(5.2, 5.2)
```

## ENUMS

```
#include <stdio.h>

typedef enum {Sa,Su} wkend;

int main() {
    wkend x, y;
}
```

An **enum**, or "enumerated type", allows us to create a custom type that should only take on the values that are enumerated in a list. First, we need to define this new enum. This involves giving the new enum a name and providing the list of possible values:

```
enum <enum_name> { <list> };
```

These "values" can be any set of symbols. In the example program below, an enum type, called `days`, is created. The possible values are the two-letter acronyms of the possible days of the week ("Mo", "Tu", "We", etc.). Internally, however, C maps these possible values to consecutive integers, starting at 0. This means, "Mo" is simply equivalent to 0, "Tu" is 1, and so on.

The syntax of declaring variables of an enum type, is as follows:

```
enum <enum_name> <variable_name>;
```

In the example below, variable `x` is declared as being of type `enum days`.

```c
#include <stdio.h>

enum days {Mo,Tu,We,Th,Fr,Sa,Su};

int main() {
    enum days x;
    x = We;
    printf("%d \n",x);
}
```

```
2
```

### 8.3.2 Enum Mapping

By default, the values defined in the enum type are mapped to consecutive integers, starting at 0. We can change this by assigning specific integer values, as illustrated in the example below. Any values that are not explicitly listed, are assigned consecutively from the previous one in the list. In the example below, the mapping is thus as follows:

Mo: 0 Tu: 3 We: 4 Th: 5 Fr: 6 Sa: 9 Su: 10

```c
#include <stdio.h>

enum days {Mo,Tu=3,We,Th,Fr,Sa=9,Su};

int main() {
    enum days x;
    x = We;
    printf("%d \n",x);
}
```

```
4
```

## FUNCTION DECLARATION

```c
int add(int x, int y);                  // Function declaration

int main() {
    int a = 10, b = 3;
    c = add(a,b+1);                     // Function call
}

int add(int x, int y) {                 // Function definition
    int sum = x + y;
    return sum;
}
```

The compiler is only interested in checking that the function is called with the correct types. As such, the function declaration does not actually need to contain the variable names of the function parameters. This means that the function declaration below is equally valid.

```c
int add(int, int);                      // Function declaration

int main() {
    int a = 10, b = 3;
    c = add(a,b+1);                     // Function call
}

int add(int x, int y) {                 // Function definition
    int sum = x + y;
    return sum;
}
```

When function declarations are used, function definitions can be anywhere in your code, even ir other files (as long as they are compiled together into one program)[1]. This is discussed in more

For now, let's focus on using `break` with loops. When using a break statement in loop constructs (for-loop, while-loop or do-while loop), the remainder of the code in the loop is skipped and the loop is exited immediately. Operation resumes after the loop statement. This is illustrated in the code below, where the for-loop does not go through all of its 10 iterations. Instead, we break out of it when `i` is equal to 3. Note that break statements work on loops, but not on if-statements.

```c
#include <stdio.h>

int main() {
    int i = 0, count = 0;
    for(i = 0; i < 10; i++) {
        if (i == 3)
            break;
        count++;
        printf("This is iteration: %d", i);
    }
    printf("Loop ran %d times", count);
}
```

```
This is iteration: 0
This is iteration: 1
This is iteration: 2
Loop ran 3 times
```

The **continue** statement is also a jump statement. As with break, when encountered inside a loop body, it skips the remainder of the code in the loop. However, it does not exit the loop completely, but goes back to checking the condition and possibly moves onto the next iteration of the loop.

The continue statement's use is limited to loops: the for-loop, while-loop, and do-while loop. It is illustrated below. Whenever `i` is even, the lines remaining in the loop body are skipped.

```c
#include <stdio.h>
int main() {
    int i;
    for (i = 0; i < 5; i++){
        if (i%2 == 0)           // Checks if i is even
            continue;
        printf("i: %d \n", i);
    }
}
```

```
i: 1
i: 3
```

```c
#include <stdio.h>
#include <stdlib.h> // Needs to be included to use exit() !

int process(int x) {
   if (x == 0)
      exit(1);          // Ends the program
   else if (x < 0)
      return -1/x;      // Exits the function process()
   else
      return 1/x;       // Exits the function process()
}

int main() {
   int a = 1, b;
   b = process(a);
   return 0;            // Exits main(), which ends the program
}
```

The example below illustrates how this loop can be useful. The code on the left uses the do-while loop. It asks the user for input[2] and continues to do so until a non-negative value was entered. The advantage is that it guarantees to always ask at least once. Compare this to the equivalent while-loop on the right.

```c
#include <stdio.h>

int main() {
    int val;
    do
        scanf("%d",&val);
    while (val < 0);
}
```

```c
#include <stdio.h>

int main() {
    int val;
    scanf("%d",&val);
    while (val < 0)
        scanf("%d",&val);
}
```

```c
#include <stdio.h>

int main() {
    int mode = 5;
    if (mode == 3)
        printf("Using low settings\n");
    elsif (mode == 5)
        printf("Using middle settings\n");
    elsif (mode == 9)
        printf("Using high settings\n");
    else
        printf("Invalid mode\n");
}
```

```
Using middle settings
```

```c
#include <stdio.h>

int main() {
    int mode = 5;
    switch (mode) {
        case 3:
            printf("Using low settings\n");
            break;
        case 5:
            printf("Using middle settings\n");
            break;
        case 9:
            printf("Using high settings\n");
            break;
        default:
            printf("Invalid mode\n");
            break;
    }
}
```

```
Using middle settings
```

**Definition**: An expression is a constant, a variable, or a meaningful (valid) combination of constants, variables, and/or other expressions, that are combined using operators. Every expression has a type and a value.

### 9.5.1 Ternary or Conditional Operator

A useful operator, offering increased code readability, is the **conditional** operator, also called the **ternary** operator. As the latter name suggests, what is unique about this operator is that it involves *three* pieces of data.

```
expr1 ? expr2 : expr3
```

The operator works by evaluating the expression ***expr1*** first. If it evaluates to true, ***expr2*** is evaluated and its result is used as the result of the overall expression. If, on the other hand, expr1 evaluates to false, then ***expr3*** is evaluated instead of *expr2* and its result used for the overall expression.

```
(5 < 2) ? 4 : 6
```

This is a single expression, built by applying the ternary operator on expressions.
(5 < 2) evaluates to false, so the value of this overall expression is 6.

| Operation | Notation | Precedence | Associativity |
|---|---|---|---|
| Primary operators | ( ) [ ]<br>*expr*++ *expr*--<br>. -> | 1 | left-to-right |
| Unary Operations | ++*expr* --*expr*<br>- !<br>* &<br>*sizeof*()<br>(*typecast*) | 2 | **right-to-left** |
| Multiplication and Division | * / % | 3 | left-to-right |
| Addition and Subtraction | + - | 4 | left-to-right |
| Relational Operators | < > <= >= | 5 | left-to-right |
| Equality Operators | == != | 6 | left-to-right |
| Logical AND | && | 7 | left-to-right |
| Logical OR | \|\| | 8 | left-to-right |
| Ternary Operator | ? : | 9 | Left-to-right |
| Assignment Operations | = += -=<br>*= /= %= | 10 | **right-to-left** |
| Comma operator | , | 11 | left-to-right |

Arithmetic

Relational

Logical